

计算机网络 Ch6 —传输层考点复习

基于廖志芳老师课件（76 页 PPT）

2026 年 6 月

目录

1	传输层概述 [p.2-6]	2
2	【重点考点】UDP 协议 vs TCP 协议对比 [p.7-15]	2
2.1	UDP (User Datagram Protocol)	2
2.2	UDP vs TCP 协议连连看对比表 [p.12]	2
3	TCP 可靠传输机制 [p.16-30]	2
3.1	TCP 头部关键字段 [p.18]	2
3.2	三次握手 (Three-Way Handshake) [p.20]	3
3.3	四次挥手 (Four-Way Handshake) [p.24]	3
3.4	TCP 流量控制 [p.26-30]	3
4	【重点考点】TCP 拥塞控制 [p.31-55]	3
4.1	拥塞控制 vs 流量控制 [p.32]	3
4.2	RTT 估算与超时重传 [p.34-37]	4
4.3	四种拥塞控制算法 [p.38-52]	4
4.3.1	1. 慢启动 (Slow Start) [p.39]	4
4.3.2	2. 拥塞避免 (Congestion Avoidance) [p.40]	4
4.3.3	3. 快重传 (Fast Retransmit) [p.42]	4
4.3.4	4. 快恢复 (Fast Recovery) [p.44]	4
4.4	TCP Tahoe vs TCP Reno [p.46]	5
4.5	AIMD 原则总结 [p.48]	5
5	本章核心考点速查	5

1 传输层概述 [p.2-6]

功能：将数据从源端口发送到目的端口——**进程到进程 (Process-to-Process)** 的通信。网络层定位到一台主机 (Host-to-Host)，传输层定位到主机上的具体进程 [p.3]。

两类传输模式：

- **可靠传输模式 (TCP)：**适合于文件传输等对通信质量有要求、不允许数据丢失的场景
- **不可靠传输模式 (UDP)：**更快捷、更轻量的端到端数据传输，适合于对通信质量要求不高、对响应速度要求高的场景，如语音对话、视频会议 [p.5]

2 【重点考点】UDP 协议 vs TCP 协议对比 [p.7-15]

2.1 UDP (User Datagram Protocol)

特点：无连接 (Connectionless)、不可靠 (Best Effort)、低开销 (8 字节头部)、支持广播和组播、无拥塞控制、无流量控制。UDP 头部 (8 字节)：源端口 (16b)、目的端口 (16b)、UDP 长度 (16b, 含头部)、校验和 (16b, 可选) [p.8]。

适用场景：DNS 查询 (端口 53)、VoIP 语音、视频流、在线游戏——可以容忍少量丢包但要求低延迟 [p.10]。

2.2 UDP vs TCP 协议连连看对比表 [p.12]

特性	UDP	TCP
连接性	无连接 (直接发送)	面向连接 (先三次握手)
可靠性	不可靠 (Best Effort)	可靠 (确认 ACK+ 超时重传)
顺序保证	不保证 (可能乱序)	保证按序交付 (序号 + 重组)
连接建立	无	三次握手 (SYN-SYN+ACK-ACK)
连接释放	无	四次挥手 (FIN-ACK-FIN-ACK)
拥塞控制	无	有 (慢启动/拥塞避免/快重传/快恢复)
流量控制	无	有 (滑动窗口 + rwnd)
头部大小	8 字节	20-60 字节 (20B 基础 + 选项)
广播/组播支持	是	否 (仅单播)
全双工	是	是
典型应用	DNS/DHCP/SNMP/VoIP/视频/在线游戏	HTTP/FTP/SMTP/Telnet/SSH/文件传输

3 TCP 可靠传输机制 [p.16-30]

3.1 TCP 头部关键字段 [p.18]

源端口 (16b)、目的端口 (16b)、序号 Seq(32b, 字节流中的第一个字节的编号)、确认号 Ack(32b, 期望收到的下一个字节的序号, 即已成功收到最后一个字节序号 +1)、数据偏移 (4b, 头部长度/4)、

标志位 (6b: URG 紧急/ACK 确认/PSH 推送/RST 复位/SYN 同步/FIN 结束)、窗口大小 (16b, 接收方剩余缓冲区空间)、校验和 (16b)、紧急指针 (16b)。

3.2 三次握手 (Three-Way Handshake) [p.20]

1. 客户端 → 服务器: SYN=1, seq=x (同步请求, 客户端初始序号)
2. 服务器 → 客户端: SYN=1, ACK=1, seq=y, ack=x+1 (服务器确认 + 同步)
3. 客户端 → 服务器: ACK=1, seq=x+1, ack=y+1 (客户端确认, 连接建立)

为什么是三次? 防止已失效的连接请求 (Request) 突然到达服务器产生死连接。两次握手可能在旧的 SYN 延迟到达时建立无意义的连接 [p.21]。

SYN 洪泛攻击 (SYN Flood) [p.22]: 攻击者发送大量 SYN 但不完成握手, 服务器为每个半连接 (SYN_RCVD 状态) 分配资源。防御: SYN Cookie——服务器不分配资源, 而是将连接信息编码在初始序号中返回。

3.3 四次挥手 (Four-Way Handshake) [p.24]

1. 客户端 → 服务器: FIN=1 (主动关闭, 客户端进入 FIN_WAIT_1)
2. 服务器 → 客户端: ACK (确认收到 FIN, 服务器进入 CLOSE_WAIT, 客户端进入 FIN_WAIT_2。服务器仍可继续发送剩余数据)
3. 服务器 → 客户端: FIN=1 (服务器也准备关闭, 进入 LAST_ACK)
4. 客户端 → 服务器: ACK (确认, 进入 TIME_WAIT 状态, 等待 2MSL 后关闭)

TIME_WAIT 状态 (2MSL 2×2 分钟): 确保最后一个 ACK 能被对方收到; 让旧连接的所有分组都从网络中消失, 防止干扰新连接 [p.25]。

3.4 TCP 流量控制 [p.26-30]

滑动窗口 (Sliding Window): 接收方通过 TCP 头部的窗口字段 (rwnd, Receiver Window) 通知发送方剩余缓冲区大小。发送窗口 = $\min(rwnd, cwnd)$ 。**零窗口探测 (Zero Window Probe):** 当 rwnd=0 时发送方定期发送探测报文。**糊涂窗口综合症 (Silly Window Syndrome):** Nagle 算法 (发送端减少小数据段) + Clark 方案 (接收端避免通告小窗口) [p.28]。

4 【重点考点】TCP 拥塞控制 [p.31-55]

4.1 拥塞控制 vs 流量控制 [p.32]

- **流量控制 (Flow Control):** 防止发送方淹没接收方——由接收方窗口 rwnd 限制
- **拥塞控制 (Congestion Control):** 防止发送方淹没网络——由拥塞窗口 cwnd 限制

发送窗口 = $\min(rwnd, cwnd)$ [p.33]。

4.2 RTT 估算与超时重传 [p.34-37]

Jacobson/Karels 算法 (SRTT + RTTVAR + RTO):

$$\text{SRTT} = (1 - \alpha)\text{SRTT} + \alpha \cdot \text{RTT}_{\text{sample}}, \quad \alpha = 1/8$$

$$\text{RTT}_{\text{VAR}} = (1 - \beta)\text{RTT}_{\text{VAR}} + \beta \cdot |\text{SRTT} - \text{RTT}_{\text{sample}}|, \quad \beta = 1/4$$

$$\text{RTO} = \text{SRTT} + 4 \cdot \text{RTT}_{\text{VAR}}$$

Karn 算法: 当发生超时重传时, 收到的 ACK 无法区分是原始报文段的确认还是重传报文段的确认, 因此重传时不采样 RTT(避免歧义)。重传超时后 RTO 加倍 (指数退避 Exponential Backoff) [p.35]。

4.3 四种拥塞控制算法 [p.38-52]

4.3.1 1. 慢启动 (Slow Start) [p.39]

- 初始 $\text{cwnd} = 1 \text{ MSS}$ (Maximum Segment Size)
- 每收到一个 ACK: $\text{cwnd} += 1 \text{ MSS}$ (即每 RTT, cwnd 翻倍——指数增长)
- 当 $\text{cwnd} \geq \text{ssthresh}$ (慢启动阈值) 时, 进入拥塞避免阶段
- 若发生超时: $\text{ssthresh} = \text{cwnd}/2$, $\text{cwnd} = 1$, 重新慢启动

4.3.2 2. 拥塞避免 (Congestion Avoidance) [p.40]

- 每 RTT 将 cwnd 增加 1 MSS (线性增长 Additive Increase)
- 若发生超时: $\text{ssthresh} = \text{cwnd}/2$, $\text{cwnd} = 1$, 重新进入慢启动

4.3.3 3. 快重传 (Fast Retransmit) [p.42]

- 接收方收到乱序报文段时, 立即发送重复 ACK (Duplicate ACK, 重复确认上一个按序到达的字节)
- 发送方收到 3 个重复 ACK 时, 认为该报文段已丢失, 不等超时, 立即重传

4.3.4 4. 快恢复 (Fast Recovery) [p.44]

- 3 个重复 ACK 触发 (非超时): $\text{ssthresh} = \text{cwnd}/2$, $\text{cwnd} = \text{ssthresh} + 3$
- 之后每收到一个重复 ACK: $\text{cwnd} += 1 \text{ MSS}$ (暂时膨胀)
- 收到新的 ACK (对重传的确认) 后: $\text{cwnd} = \text{ssthresh}$, 进入拥塞避免阶段
- 关键: 快重传 + 快恢复配合使用, 避免单次丢包就从慢启动重新开始

4.4 TCP Tahoe vs TCP Reno [p.46]

- **TCP Tahoe:** 慢启动 + 拥塞避免 + 快重传。收到 3 个重复 ACK 后, $ssthresh=cwnd/2$, $cwnd=1$, 重新进入慢启动。不区分超时和 3 个重复 ACK
- **TCP Reno:** Tahoe + 快恢复。收到 3 个重复 ACK 后, $ssthresh=cwnd/2$, $cwnd=ssthresh+3$, 进入快恢复而非慢启动。更高效——避免不必要的慢启动

4.5 AIMD 原则总结 [p.48]

TCP 拥塞控制的 AIMD(Additive Increase, Multiplicative Decrease) 原则:

- 加性增 (AI): 拥塞避免阶段每 RTT 增加 1 MSS
- 乘性减 (MD): 检测到丢包时 $cwnd$ 减半

5 本章核心考点速查

1. **UDP vs TCP 协议连连看对比表 [p.12]:** 连接性/可靠性/顺序/拥塞控制/流量控制/头部开销/典型应用
2. **TCP 三次握手 [p.20]:** $SYN \rightarrow SYN+ACK \rightarrow ACK$ 。为什么三次? SYN Flood 攻击与 SYN Cookie 防御
3. **TCP 四次挥手 [p.24]:** $FIN \rightarrow ACK \rightarrow FIN \rightarrow ACK$ 。TIME_WAIT 2MSL 原因
4. **TCP 流量控制 [p.26-30]:** 滑动窗口 + $rwnd$ + 零窗口探测 + 糊涂窗口 (Nagle/Clark)
5. **RTT 估算 [p.34-37]:** Jacobson 算法 SRTT/RTTVAR/RTO 公式。Karn 算法 (重传不采样)
6. **拥塞控制四种算法 [p.38-52]:** 慢启动 (指数增, $cwnd$ 翻倍) $\rightarrow$ 拥塞避免 (线性增, 每 RTT+1) $\rightarrow$ 快重传 (3 重复 ACK 立即重传) $\rightarrow$ 快恢复 ($ssthresh=cwnd/2$, $cwnd=ssthresh+3$)
7. **Tahoe vs Reno [p.46]:** Reno 多了快恢复
8. **AIMD 原则 [p.48]:** 加性增 (每 RTT+1) 乘性减 (丢包 $\div 2$)